OS

Paturu Naga Jignas

PES2UG24AM110

github.com/JignasP/Version-Control-System

Phase 1.

```
PS C:\Users\Jignas\Desktop\os> wsl bash -lc 'cd /mnt/c/Users/J
/Version-Control-System; make test_objects; ./test_objects'
Stored blob with hash: d58213f5dbe0629b5c2fa28e5c7d4213ea09227e
4b9aafc12
Object stored at: .pes/objects/d5/8213f5dbe0629b5c2fa28e5c7d421
be9db5e5c4b9aafc12
PASS: blob storage
PASS: deduplication
PASS: integrity check

All Phase 1 tests passed.
be9db5e5c4b9aafc12
PASS: blob storage
PASS: deduplication
PASS: integrity check

be9db5e5c4b9aafc12
PASS: blob storage
PASS: deduplication
be9db5e5c4b9aafc12
PASS: blob storage
be9db5e5c4b9aafc12
```

```
PS C:\Users\Jignas\Desktop\os> wsl bash -lc 'cd /mnt/c/Users/J
/Version-Control-System; find .pes/objects -type f | sort'
.pes/objects/25/ef1fa07ea68a52f800dc80756ee6b7ae34b337afedb9b4
.pes/objects/2a/594d39232787fba8eb7287418aec99c8fc2ecdaf5aaf2e
.pes/objects/d5/8213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe
```

Phase 2.

```
PS C:\Users\Jignas\Desktop\os> wsl
root@DESKTOP-PLUV6D6:/mnt/c/Users/Jignas/Desktop/os# cd /mnt/c/
sktop/os/Version-Control-System; make test_tree >/tmp/phase2_bu
/test_tree
Serialized tree: 139 bytes
PASS: tree serialize/parse roundtrip
PASS: tree deterministic serialization
```

```
root@DESKTOP-PLUV6D6:/mnt/c/Users/Jignas/Desktop/os/Version-Cor
pdir=$(mktemp -d); cd "$tmpdir"; /mnt/c/Users/Jignas/Desktop/os
l-System/pes init >/dev/null; echo hello > a.txt; /mnt/c/Users/
os/Version-Control-System/pes add a.txt >/dev/null; /mnt/c/User
p/os/Version-Control-System/pes commit -m x >/dev/null; treefil
bjects -type f | while read -r f; do if xxd -l 4 -p "$f" | grep
en echo "$f"; break; fi; done); echo "$treefile"; xxd "$treefil
.pes/objects/37/1aac45c3ba401d13c2ce98c9537b4e53c6049b950bf8a33

00000000: 7472 6565 2034 3500 3130 3036 3434 2061  tree 45.1006
```

Phase 3.

```
PS C:\Users\Jignas\Desktop\os> wsl
root@DESKTOP-PLUV6D6:/mnt/c/Users/Jignas/Desktop/os# cd /mnt/c/Users/Jignas/De
top/os/Version-Control-System; make pes >/tmp/phase3_build.log 2>&1; rm -rf .p
 file1.txt file2.txt; ./pes init; echo hello > file1.txt; echo world > file2.t
; ./pes add file1.txt file2.txt; ./pes status; echo '--- INDEX ---'; cat .pes/
dex
Initialized empty PES repository in .pes/
Staged changes:
  staged:     file1.txt
  staged:     file2.txt

Unstaged changes:
  (nothing to show)

Untracked files:
  untracked:  .gitignore
  untracked:  bye.txt
  untracked:  commit.c
  untracked:  commit.h
  untracked:  file.txt
  untracked:  hello.txt
  untracked:  index.c
  untracked:  index.h
  untracked:  Makefile
  untracked:  object.c
  untracked:  pes.c
  untracked:  pes.h
  untracked:  README.md
  untracked:  test_objects
  untracked:  test_objects.c
  untracked:  test_sequence.sh
  untracked:  test_tree
```

Phase 4

```
PS C:\Users\Jignas\Desktop\os> wsl
root@DESKTOP-PLUV6D6:/mnt/c/Users/Jignas/Desktop/os# tmpdir=$(mkte
mpdir"; /mnt/c/Users/Jignas/Desktop/os/Version-Control-System/pes
l; echo Hello > hello.txt; /mnt/c/Users/Jignas/Desktop/os/Version-
/pes add hello.txt >/dev/null; /mnt/c/Users/Jignas/Desktop/os/Vers
stem/pes commit -m 'Initial commit'; echo World >> hello.txt; /mnt
s/Desktop/os/Version-Control-System/pes add hello.txt >/dev/null;
ignas/Desktop/os/Version-Control-System/pes commit -m 'Add world';
 > bye.txt; /mnt/c/Users/Jignas/Desktop/os/Version-Control-System/
t >/dev/null; /mnt/c/Users/Jignas/Desktop/os/Version-Control-Syste
m 'Add farewell'; /mnt/c/Users/Jignas/Desktop/os/Version-Control-S
Committed: f8c82510423d... Initial commit
Committed: e2c8c815e52a... Add world
Committed: 3f541b42426e... Add farewell
commit 3f541b42426e3e4c5f0d806c105a33005c3cee1ceab7b7441114dafe4d9
Author: PES User <pes@localhost>
Date:   1776404707

    Add farewell

commit e2c8c815e52a4cf29562c49122eb64d9dbe96029e32bf0299b17b09537c
Author: PES User <pes@localhost>
Date:   1776404707

    Add worldell
```

```
root@DESKTOP-PLUV6D6:/tmp/tmp.CWZ3qdReFU# find .pes -type f | sort;
cat .pes/refs/heads/main; echo '---'; cat .pes/HEAD
.pes/HEAD
.pes/index
.pes/objects/05/c8d777d0be85748b6788d1b5caf7d69754e6e68060c265ca21cb
.pes/objects/1c/cd295330bb5eb8f89951f8ca6c1890333b848d115a7d5ecf4510
.pes/objects/3f/541b42426e3e4c5f0d806c105a33005c3cee1ceab7b7441114da
.pes/objects/d8/f28203071e10a3bde605928368bc5b51871287a867d8d1038296
.pes/objects/e2/c8c815e52a4cf29562c49122eb64d9dbe96029e32bf0299b17b0
.pes/objects/f8/c82510423d1358ea06506f0ecdbbed804b27af05d21ebe0a7253
.pes/refs/heads/main
---
3f541b42426e3e4c5f0d806c105a33005c3cee1ceab7b7441114dafe4d9101d8
```

Phase 5 and 6:

Analysis Questions

Q5.1: A branch in Git is just a file in .git/refs/heads/ containing a commit hash. Creating a branch is creating a file. Given this, how would you implement pes checkout <branch> — what files need to change in .pes/, and what must happen to the working directory? What makes this operation complex?

A branch in Git is simply a reference file that stores a commit hash, so implementing pes checkout <branch> involves updating both internal references and the working directory. First, the .pes/HEAD file must be updated to point to the new branch (e.g., ref: refs/heads/<branch>), and the corresponding branch file in .pes/refs/heads/ must be read to obtain the commit hash. Then, the system must load that commit, retrieve its associated tree, and reconstruct the working directory by writing all files from the tree (via blob objects) and removing any files that are not present in the target branch. The .pes/index must also be updated to match the new commit's state. This operation is complex because it must handle uncommitted changes that could be overwritten, resolve potential file conflicts, correctly rebuild directory structures, and ensure that the HEAD, index, and working directory remain consistent throughout the process.

Q5.2: When switching branches, the working directory must be updated to match the target branch's tree. If the user has uncommitted changes to a tracked file, and that file differs between branches, checkout must refuse. Describe how you would detect this "dirty working directory" conflict using only the index and the object store.

To detect a "dirty working directory" conflict during pes checkout, we can compare the current working directory against the index and the target branch's tree using the object store. For each file tracked in the index, we first recompute its hash from the working directory and compare it with the stored hash in the index; if they diƯer, the file has uncommitted changes. Next, we load the target branch's commit and its tree, and check whether the same file exists there with a diƯerent hash than in the current index. If both

conditions are true—(1) the file is modified in the working directory compared to the index, and (2) the file differs between the current index and the target branch's tree—then a conflict exists, and checkout must refuse to proceed. This approach relies only on hashing and comparisons with stored object hashes, without needing additional metadata.

Q5.3: "Detached HEAD" means HEAD contains a commit hash directly instead of a branch reference. What happens if you make commits in this state? How could a user recover those commits?

In a detached HEAD state, HEAD points directly to a specific commit instead of a branch, so any new commits made will not belong to any branch. These commits are still created and stored in the object database, but since no branch reference points to them, they become "dangling" and can be lost once they are no longer reachable. To recover such commits, the user can create a new branch pointing to the current commit (e.g., by updating a file in .pes/refs/heads/ to the commit hash), thereby attaching the detached commits to a branch and making them part of the normal history.

Q6.1: Over time, the object store accumulates unreachable objects — blobs, trees, or commits that no branch points to (directly or transitively). Describe an algorithm to find and delete these objects. What data structure would you use to track "reachable" hashes efficiently?

For a repository with 100,000 commits and 50 branches, estimate how many objects you'd need to visit.
Garbage collection removes objects that are no longer reachable from any commit or reference, freeing storage

Q6.2: Why is it dangerous to run garbage collection concurrently with a commit operation?

Describe a race condition where GC could delete an object that a concurrent commit is about to reference. How does Git's real GC avoid this?

To perform garbage collection, we first identify all reachable objects by starting from every branch reference in .pes/refs/heads/ and traversing the commit graph. For each branch, we follow its commit hash, then recursively visit its parent commits and associated tree objects, and from each tree, traverse all referenced blobs and subtrees. During this traversal, we store every visited hash in a hash set (or hash table), which allows eƱicient O(1) lookup to mark objects as reachable. After this marking phase, we scan the entire .pes/objects/ directory and delete any object whose hash is not present in the reachable set, as these are unreachable (garbage). For a repository with 100,000 commits and 50 branches, although there are 50 starting points, many commits are shared across branches, so the traversal will typically visit on the order of ~100,000 commits plus their associated trees and blobs, rather than 50×100,000. Including trees and blobs, the total number of visited objects may be a few times larger, but still linear in the size of the repository, making the algorithm eƱicient.